



Programación de aplicaciones en Spark

Rafael Pastor Vargas
Andrés Duque Fernández
SCC-ETSI Informática-UNED

PROGRAMACIÓN DE APLICACIONES EN SPARK

- Entornos de programación de aplicaciones Spark
- Desarrollo y ejecución de una aplicación Spark.
- Creación de sesiones: objetos SparkSession/SparkContext.
- Inicialización de Spark en varios lenguajes: Scala/Python.
- Cargar y guardar datos en Spark.
- Colecciones de datos: Resilient Distributed Dataset, DataFrames y Datasets.
- Paralelización de las colecciones de datos.
- Operaciones sobre RDD.
- El API de Datasets/DataFrames.

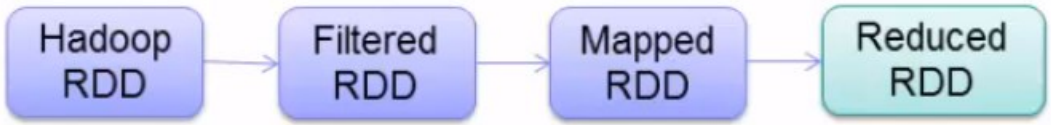
SPARK CORE: RESILIENT DISTRIBUTED DATASET

- Son estructuras de datos que contienen elementos que pueden ser manipulados en paralelo
  - Distribuidos en un conjunto de máquinas/nodos de computación
  - Las transformaciones de datos se pueden recrear/regenerar en caso de fallo de nodo/máquina o corrupción de memoria
- Abstracciones de bajo nivel en Spark 2.x
  - Datasets, Dataframes (API de programación, abstracciones de alto nivel)
    - dataset.rdd()
    - SparkSession.createDataset(rdd), SparkSession.createDataFrame(rdd)
  - **Todos los datos** son representados como RDDs
- RDDs en Scala → Scala Collection Library

  https://www.scala-lang.org/api/current/scala/collection/index.html

RDD: FUNDAMENTOS (1/12)

- Dos tipos de operaciones con RDD
- Transformaciones
  - Creación del DAG (Direct Acyclic Graph)
  - Evaluaciones "en diferido/vagas/perezosas" (lazy)
  - No devuelven **valores**
- Acciones
  - Ejecutan las transformaciones y la propia acción
  - Devuelven un valor
- Son tolerantes a fallos
- Se pueden cachear
- Ejemplo de flujo



RDD: FUNDAMENTOS (2/12)

- Inmutables
- Existen tres métodos para crear RDDs
  - Paralelizar una colección ya existente
  - Referenciar un Dataset
  - Usar una transformación de un RDD existente
- Cuando se crea un RDD se crea un DAG
  - Las transformaciones actualizan el DAG pero no se "ejecuta" nada
- Los datasets pueden usarse desde cualquier tipo de almacenamiento soportado por Hadoop: HDFS, Cassandra, Hbase, Amazon S3, etc.
- Tipos de ficheros soportados
  - Texto
  - Ficheros secuenciales (SequenceFiles)
  - Hadoop (InputFormat)

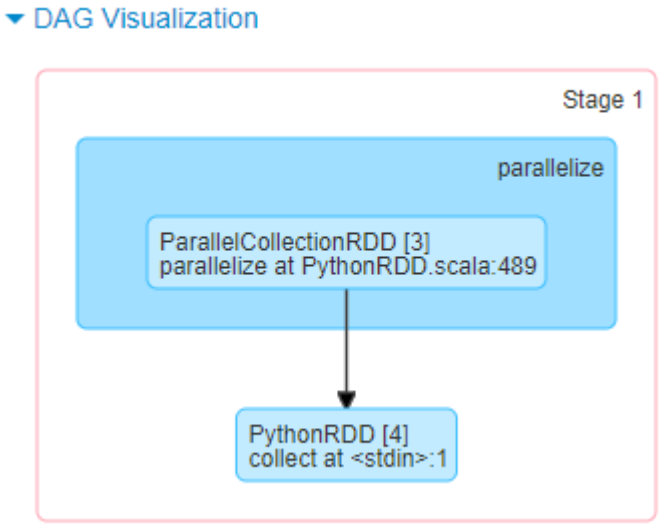
RDD: FUNDAMENTOS (3/12)

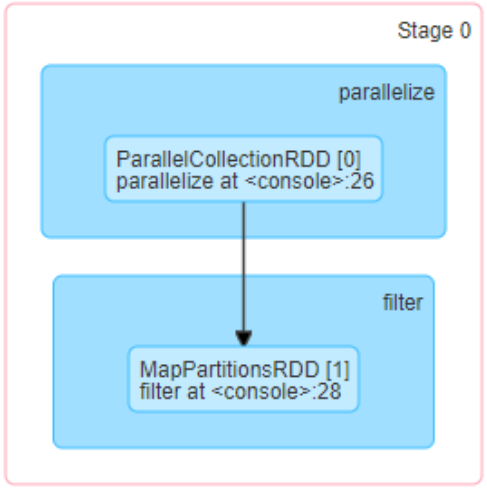
- Ejemplo de creación de un RDD a partir de una colección existente

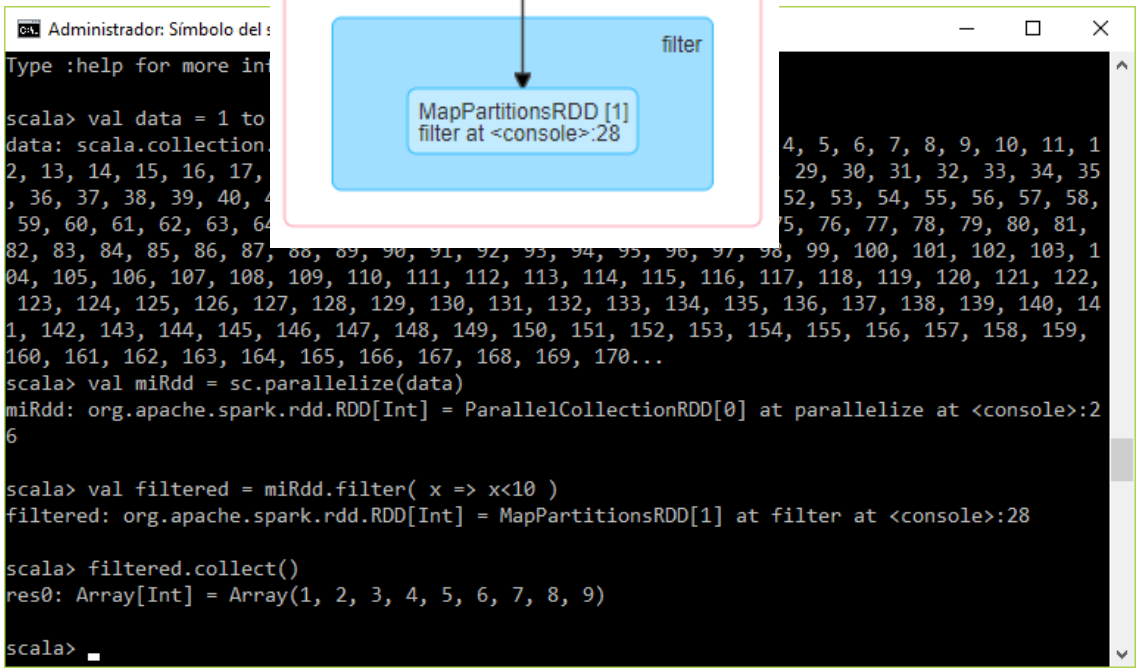
```
# Spark-Shell
val data = 1 to 10000
val miRdd = sc.parallelize(data)
val filtered = miRdd.filter( x => x<10 )
filtered.collect()
# Pyspark
data = range(1,10001)
miRdd = sc.parallelize(data)
filtered = miRdd.filter(lambda x: x<10)
filtered.collect()
```

RDD: FUNDAMENTOS (4/12)

- DAG
  - Se puede ver el DAG asociado a un RDD en particular
    - toDebugString()

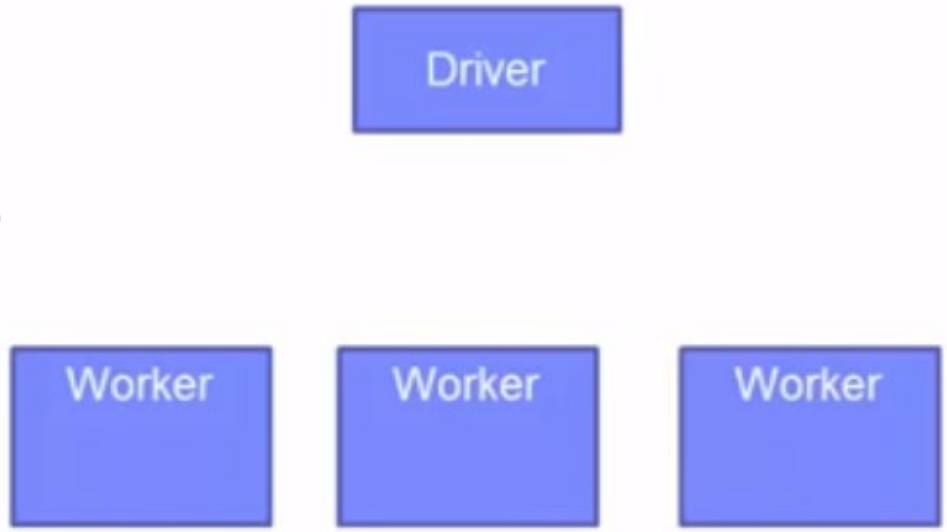






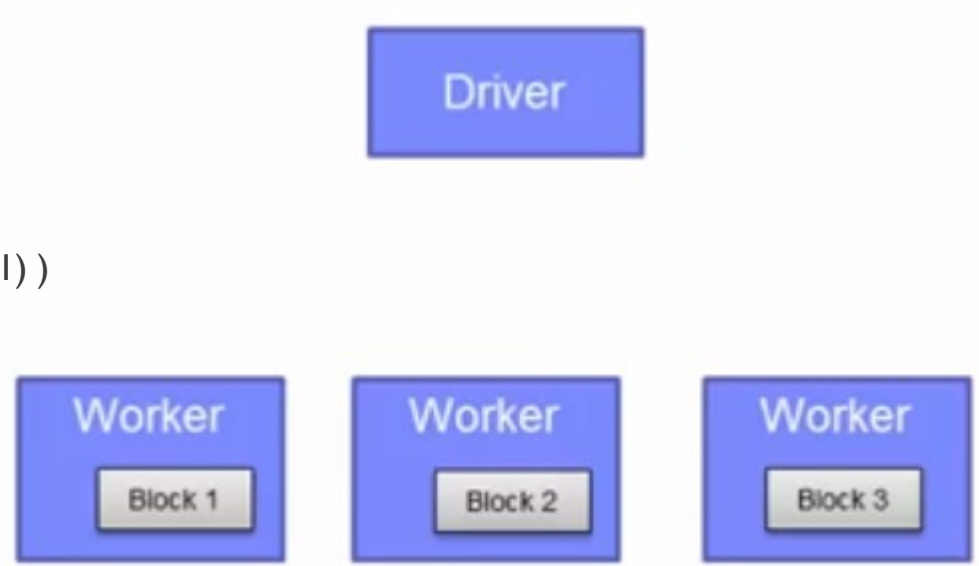
RDD: FUNDAMENTOS (5/12)

```
// Creacion del RDD con el log de error
val logFile = sc.textFile("hdfs://...")
// Transformaciones
val errors = logFile.filter(_.startsWith("ERROR"))
val messages = errors.map( _.split("\t")).map( r => r(1) )
// Caching
messages.cache()
//Actions
messages.filter(_.contains("mysql")).count()
messages.filter(_.contains("php")).count()
```



RDD: FUNDAMENTOS (6/12)

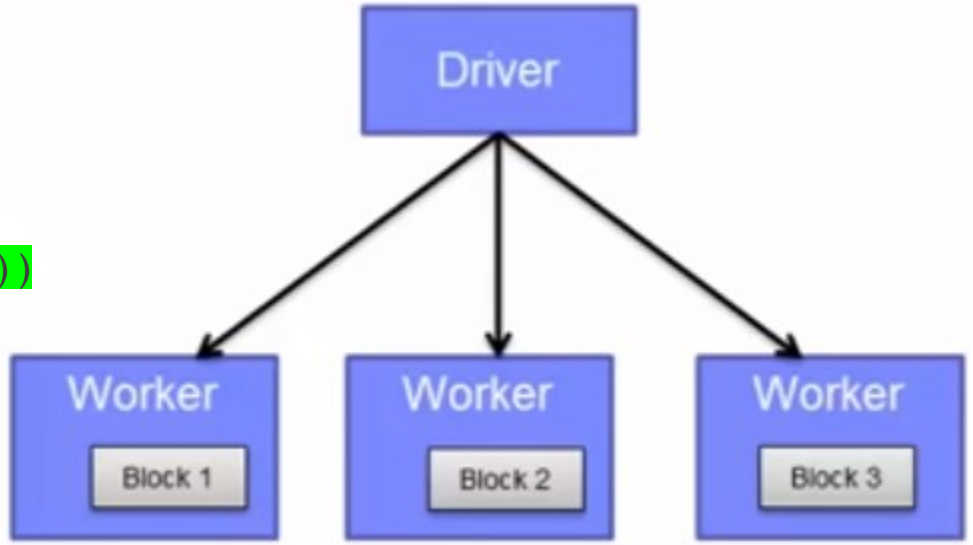
```
// Creacion del RDD con el log de error
val logFile = sc.textFile("hdfs://...")
// Transformaciones
val errors = logFile.filter(_.startsWith("ERROR"))
val messages = errors.map( _.split("\t")).map( r => r(1) )
// Caching
messages.cache()
//Actions
messages.filter(_.contains("mysql")).count()
messages.filter(_.contains("php")).count()
```



Los datos se particionan en bloques y se envían a los ejecutores

RDD: FUNDAMENTOS (7/12)

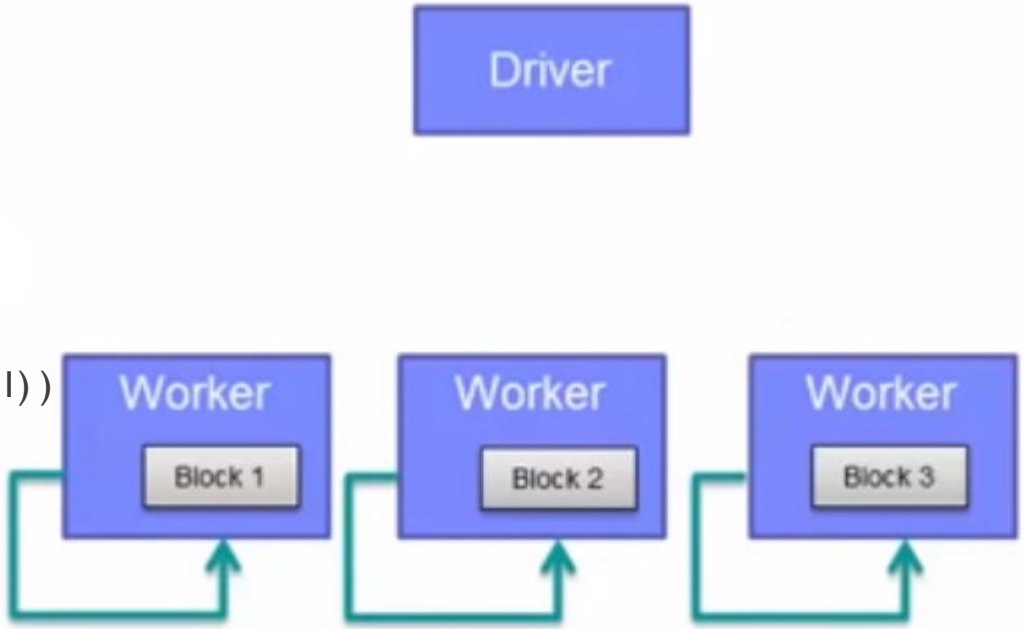
```
// Creacion del RDD con el log de error
val logFile = sc.textFile("hdfs://...")
// Transformaciones
val errors = logFile.filter(_.startsWith("ERROR"))
val messages = errors.map( _.split("\t")).map( r => r(1) )
// Caching
messages.cache()
//Actions
messages.filter(_.contains("mysql")).count()
messages.filter(_.contains("php")).count()
```



El driver envía el código a ejecutar en cada trabajador por los ejecutores

RDD: FUNDAMENTOS (8/12)

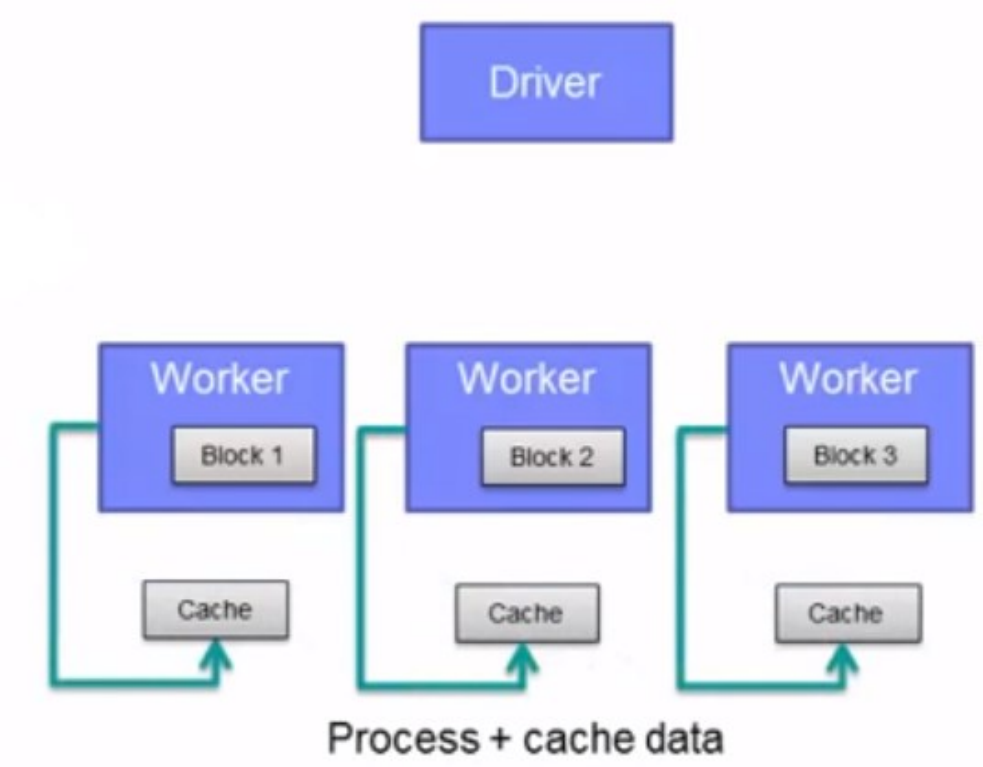
```
// Creacion del RDD con el log de error
val logFile = sc.textFile("hdfs://...")
// Transformaciones
val errors = logFile.filter(_.startsWith("ERROR"))
val messages = errors.map( _.split("\t")).map( r => r(1) )
// Caching
messages.cache()
//Actions
messages.filter(_.contains("mysql")).count()
messages.filter(_.contains("php")).count()
```



Se leen los bloques HDFS

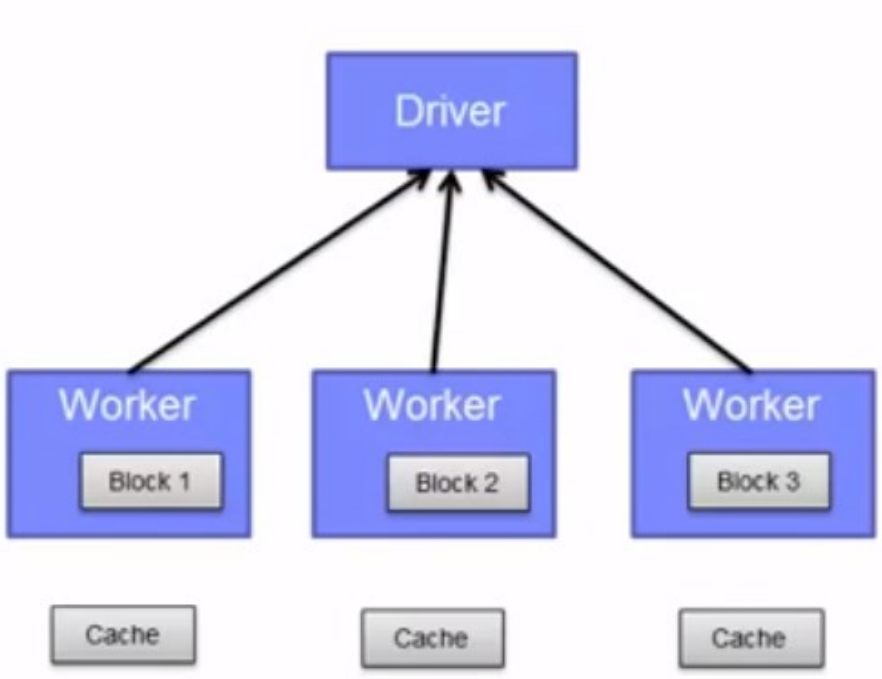
RDD: FUNDAMENTOS (9/12)

```
// Creacion del RDD con el log de error
val logFile = sc.textFile("hdfs://...")
// Transformaciones
val errors = logFile.filter(_.startsWith("ERROR"))
val messages = errors.map( _.split("\t")).map( r =>
// Caching
messages.cache()
//Actions
messages.filter(_.contains("mysql")).count()
messages.filter(_.contains("php")).count()
```



RDD: FUNDAMENTOS (10/12)

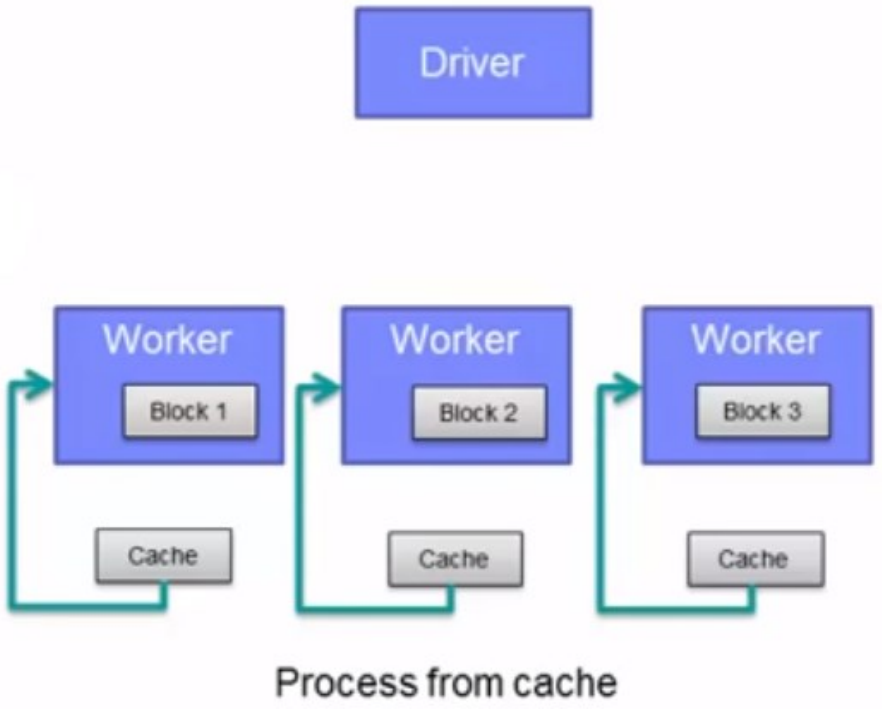
```
// Creacion del RDD con el log de error
val logFile = sc.textFile("hdfs://...")
// Transformaciones
val errors = logFile.filter(_.startsWith("ERROR"))
val messages = errors.map( _.split("\t")).map( r => r(1) )
// Caching
messages.cache()
//Actions
messages.filter(_.contains("mysql")).count()
messages.filter(_.contains("php")).count()
```



Los datos calculados en la acción se envían al driver

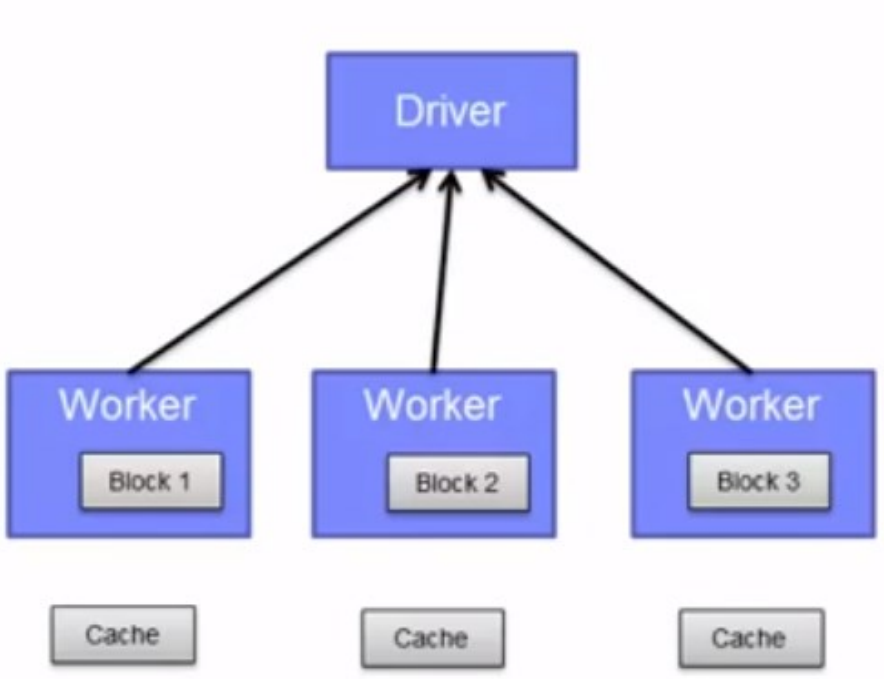
RDD: FUNDAMENTOS (11/12)

```
// Creacion del RDD con el log de error
val logFile = sc.textFile("hdfs://...")
// Transformaciones
val errors = logFile.filter(_.startsWith("ERROR"))
val messages = errors.map( _.split("\t")).map( r => r(1) )
// Caching
messages.cache()
//Actions
messages.filter(_.contains("mysql")).count()
messages.filter(_.contains("php")).count()
```



RDD: FUNDAMENTOS (12/12)

```
// Creacion del RDD con el log de error
val logFile = sc.textFile("hdfs://...")
// Transformaciones
val errors = logFile.filter(_.startsWith("ERROR"))
val messages = errors.map( _.split("\t")).map( r => r(1) )
// Caching
messages.cache()
//Actions
messages.filter(_.contains("mysql")).count()
messages.filter(_.contains("php")).count()
```



Los datos calculados en la acción se envían al driver (sin leer otra vez los bloques HDFS)

https://spark.apache.org/docs/latest/rdd-programming-guide.html#transformations

TRANSFORMACIONES (1/2)

Transformation	Meaning
map(*func*)	Return a new distributed dataset formed by passing each element of the source through a function *func*.
filter(*func*)	Return a new dataset formed by selecting those elements of the source on which *func* returns true.
flatMap(*func*)	Similar to map, but each input item can be mapped to 0 or more output items (so *func* should return a Seq rather than a single item).
mapPartitions(*func*)	Similar to map, but runs separately on each partition (block) of the RDD, so *func* must be of type Iterator<T> => Iterator<U> when running on an RDD of type T.
mapPartitionsWithIndex(*func*)	Similar to mapPartitions, but also provides *func* with an integer value representing the index of the partition, so *func* must be of type (Int, Iterator<T>) => Iterator<U> when running on an RDD of type T.
sample(*withReplacement*, *fraction*, *seed*)	Sample a fraction *fraction* of the data, with or without replacement, using a given random number generator seed.
union(*otherDataset*)	Return a new dataset that contains the union of the elements in the source dataset and the argument.
intersection(*otherDataset*)	Return a new RDD that contains the intersection of elements in the source dataset and the argument.
distinct([*numTasks*]))	Return a new dataset that contains the distinct elements of the source dataset.
groupByKey([*numTasks*])	When called on a dataset of (K, V) pairs, returns a dataset of (K, Iterable<V>) pairs. **Note:** If you are grouping in order to perform an aggregation (such as a sum or average) over each key, using `reduceByKey` or `aggregateByKey` will yield much better performance. **Note:** By default, the level of parallelism in the output depends on the number of partitions of the parent RDD. You can pass an optional `numTasks` argument to set a different number of tasks.
reduceByKey(*func*, [*numTasks*])	When called on a dataset of (K, V) pairs, returns a dataset of (K, V) pairs where the values for each key are aggregated using the given reduce function *func*, which must be of type (V,V) => V. Like in `groupByKey`, the number of reduce tasks is configurable through an optional second argument.

https://spark.apache.org/docs/latest/rdd-programming-guide.html#transformations

TRANSFORMACIONES (2/2)

Transformation	Meaning
aggregateByKey(*zeroValue*)(*seqOp*, *combOp*, [*numTasks*])	When called on a dataset of (K, V) pairs, returns a dataset of (K, U) pairs where the values for each key are aggregated using the given combine functions and a neutral "zero" value. Allows an aggregated value type that is different than the input value type, while avoiding unnecessary allocations. Like in `groupByKey`, the number of reduce tasks is configurable through an optional second argument.
sortByKey([*ascending*], [*numTasks*])	When called on a dataset of (K, V) pairs where K implements Ordered, returns a dataset of (K, V) pairs sorted by keys in ascending or descending order, as specified in the boolean `ascending` argument.
join(*otherDataset*, [*numTasks*])	When called on datasets of type (K, V) and (K, W), returns a dataset of (K, (V, W)) pairs with all pairs of elements for each key. Outer joins are supported through `leftOuterJoin`, `rightOuterJoin`, and `fullOuterJoin`.
cogroup(*otherDataset*, [*numTasks*])	When called on datasets of type (K, V) and (K, W), returns a dataset of (K, (Iterable<V>, Iterable<W>)) tuples. This operation is also called `groupWith`.
cartesian(*otherDataset*)	When called on datasets of types T and U, returns a dataset of (T, U) pairs (all pairs of elements).
pipe(*command*, *[envVars]*)	Pipe each partition of the RDD through a shell command, e.g. a Perl or bash script. RDD elements are written to the process's stdin and lines output to its stdout are returned as an RDD of strings.
coalesce(*numPartitions*)	Decrease the number of partitions in the RDD to numPartitions. Useful for running operations more efficiently after filtering down a large dataset.
repartition(*numPartitions*)	Reshuffle the data in the RDD randomly to create either more or fewer partitions and balance it across them. This always shuffles all data over the network.
repartitionAndSortWithinPartitions(*partitioner*)	Repartition the RDD according to the given partitioner and, within each resulting partition, sort records by their keys. This is more efficient than calling `repartition` and then sorting within each partition because it can push the sorting down into the shuffle machinery.

https://spark.apache.org/docs/latest/rdd-programming-guide.html#actions

ACCIONES

Actions

The following table lists some of the common actions supported by Spark. Refer to the RDD API doc (Scala, Java, Python, R)

and pair RDD functions doc (Scala, Java) for details.

Action	Meaning
reduce(*func*)	Aggregate the elements of the dataset using a function *func* (which takes two arguments and returns one). The function should be commutative and associative so that it can be computed correctly in parallel.
collect()	Return all the elements of the dataset as an array at the driver program. This is usually useful after a filter or other operation that returns a sufficiently small subset of the data.
count()	Return the number of elements in the dataset.
first()	Return the first element of the dataset (similar to take(1)).
take(*n*)	Return an array with the first *n* elements of the dataset.
takeSample(*withReplacement*, *num*, [*seed*])	Return an array with a random sample of *num* elements of the dataset, with or without replacement, optionally pre-specifying a random number generator seed.
takeOrdered(*n*, *[ordering]*)	Return the first *n* elements of the RDD using either their natural order or a custom comparator.
saveAsTextFile(*path*)	Write the elements of the dataset as a text file (or set of text files) in a given directory in the local filesystem, HDFS or any other Hadoop-supported file system. Spark will call toString on each element to convert it to a line of text in the file.
saveAsSequenceFile(*path*) (Java and Scala)	Write the elements of the dataset as a Hadoop SequenceFile in a given path in the local filesystem, HDFS or any other Hadoop-supported file system. This is available on RDDs of key-value pairs that implement Hadoop's Writable interface. In Scala, it is also available on types that are implicitly convertible to Writable (Spark includes conversions for basic types like Int, Double, String, etc).
saveAsObjectFile(*path*) (Java and Scala)	Write the elements of the dataset in a simple format using Java serialization, which can then be loaded using `SparkContext.objectFile()`.
countByKey()	Only available on RDDs of type (K, V). Returns a hashmap of (K, Int) pairs with the count of each key.
foreach(*func*)	Run a function *func* on each element of the dataset. This is usually done for side effects such as updating an Accumulator or interacting with external storage systems. **Note**: modifying variables other than Accumulators outside of the `foreach()` may result in undefined behavior. See Understanding closures for more details.

The Spark RDD API also exposes asynchronous versions of some actions, like `foreachAsync` for `foreach`, which immediately return a `FutureAction` to the caller instead of blocking on completion of the action. This can be used to manage or wait for the asynchronous execution of the action.

PERSISTENCIA Y RDDS

- Cada nodo almacena las particiones en la memoria caché (cálculos en memoria)
- Reutilización de las particiones en otras acciones sobre el RDD (rapidez y eficiencia)
- Dos métodos: persist() | cache() →persist() + MEMORY_ONLY

Storage Level	Meaning
MEMORY_ONLY	Store RDD as deserialized Java objects in the JVM. If the RDD does not fit in memory, some partitions will not be cached and will be recomputed on the fly each time they're needed. This is the default level.
MEMORY_AND_DISK	Store RDD as deserialized Java objects in the JVM. If the RDD does not fit in memory, store the partitions that don't fit on disk, and read them from there when they're needed.
MEMORY_ONLY_SER (Java and Scala)	Store RDD as *serialized* Java objects (one byte array per partition). This is generally more space-efficient than deserialized objects, especially when using a fast serializer, but more CPU-intensive to read.
MEMORY_AND_DISK_SER (Java and Scala)	Similar to MEMORY_ONLY_SER, but spill partitions that don't fit in memory to disk instead of recomputing them on the fly each time they're needed.
DISK_ONLY	Store the RDD partitions only on disk.
MEMORY_ONLY_2, MEMORY_AND_DISK_2, etc.	Same as the levels above, but replicate each partition on two cluster nodes.
OFF_HEAP (experimental)	Similar to MEMORY_ONLY_SER, but store the data in off-heap memory. This requires off-heap memory to be enabled.

RECOMENDACIONES (IBM)

Which storage level to choose?

- If your RDDs fit comfortably with the default storage level (MEMORY_ONLY), leave them that way. This is the most CPU-efficient option, allowing operations on the RDDs to run as fast as possible.
- If not, try using MEMORY_ONLY_SER and selecting a fast serialization library to make the objects much more space-efficient, but still reasonably fast to access.
- Don't spill to disk unless the functions that computed your datasets are expensive, or they filter a large amount of the data. Otherwise, recomputing a partition may be as fast as reading it from disk.
- Use the replicated storage levels if you want fast fault recovery (e.g. if using Spark to serve requests from a web application). *All* the storage levels provide full fault tolerance by recomputing lost data, but the replicated ones let you continue running tasks on the RDD without waiting to recompute a lost partition.
- In environments with high amounts of memory or multiple applications, the experimental OFF_HEAP mode has several advantages:
  - It allows multiple executors to share the same pool of memory in Tachyon.
  - It significantly reduces garbage collection costs.
  - Cached data is not lost if individual executors crash.

ACCUMULATORS Y VARIABLES BROADCAST

- El driver envía una copia separada de las variables a los nodos.
- Implementar el concepto de estado compartido (shared state).
  - Broadcast variable. La variable se envía una sola vez a todos los nodos
    - Es de solo lectura (inmutable)
    - sc.broadcast(value) → devuelve la referencia a la variable
  - Accumulator.
    - Contadores y sumas (operaciones de agregación): add() | +=
    - Solo el driver puede leer el valor de la variable acumulador
    - Solo los nodos pueden agregar información al acumulador (sumas)
    - Soporte para tipos numéricos (aunque se pueden implementar tipos específicos)

COMO FUNCIONA? (1/2)

- Spark-Shell

```
val accum = sc.longAccumulator("My Accumulator")
sc.parallelize(Array(1, 2, 3, 4)).foreach(x => accum.add(x))
accum.value
```

- Pyspark

```
accum = sc.accumulator("test",0)
accum
sc.parallelize([1, 2, 3, 4]).foreach(lambda x: accum.add(x))
accum.value
```

COMO FUNCIONA? (2/2)

Accumulators	Value
Accumulable	10
My Accumulator	

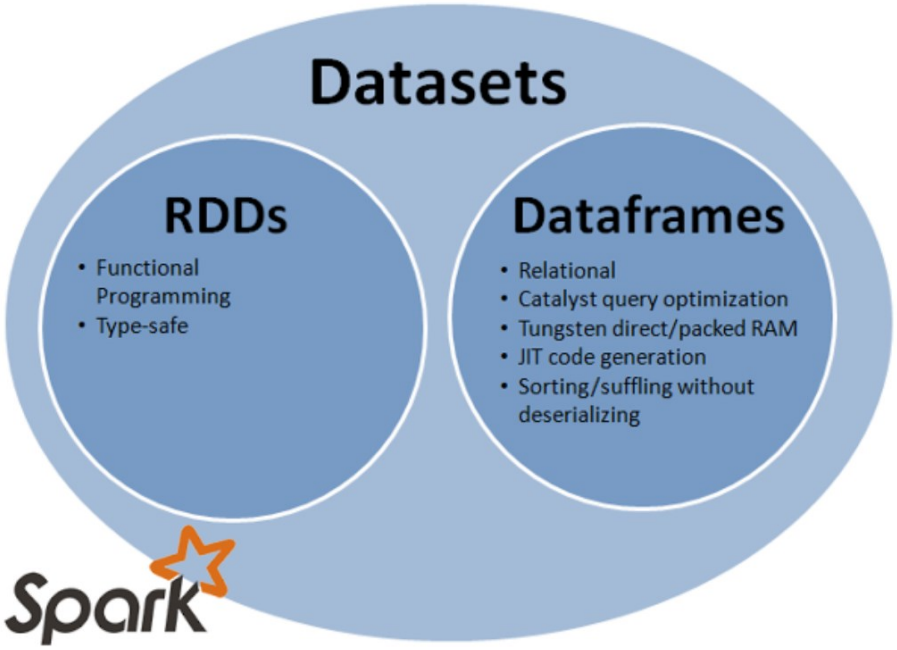
Tasks (4)

Index ▲	ID	Attempt	Status	Locality Level	Executor ID / Host
0	0	0	SUCCESS	PROCESS_LOCAL	driver / localhost
1	1	0	SUCCESS	PROCESS_LOCAL	driver / localhost
2	2	0	SUCCESS	PROCESS_LOCAL	driver / localhost
3	3	0	SUCCESS	PROCESS_LOCAL	driver / localhost

Launch Time	Duration	GC Time	Accumulators
2018/01/22 17:44:25	45 ms		My Accumulator: 1
2018/01/22 17:44:25	48 ms		My Accumulator: 2
2018/01/22 17:44:25	37 ms		My Accumulator: 3
2018/01/22 17:44:25	31 ms		My Accumulator: 4

DATAFRAMES/DATASETS (1/2)

- Desventajas del API de RDD
  - % Alto de datos estructurados/semiestructurados (JSON, CSV, Avro, Parquet, Hive) → Uso intensivo de tuplas en RDDs (_1, _2, [0], [1]…)
  - Las transformaciones funcionales no son intuitivas (map/reduce, etc.)

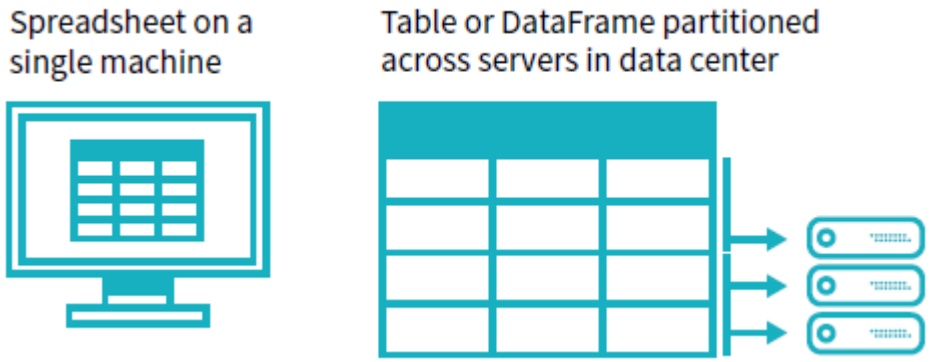



```
pdata.map(lambda x: (x.dept, [x.age, 1])) \
  .reduceByKey(lambda x, y: [x[0] + y[0], x[1] + y[1]]) \
  .map(lambda x: [x[0], x[1][0] / x[1][1]]) \
  .collect()
```

```
data.groupBy("dept").avg("age")
```

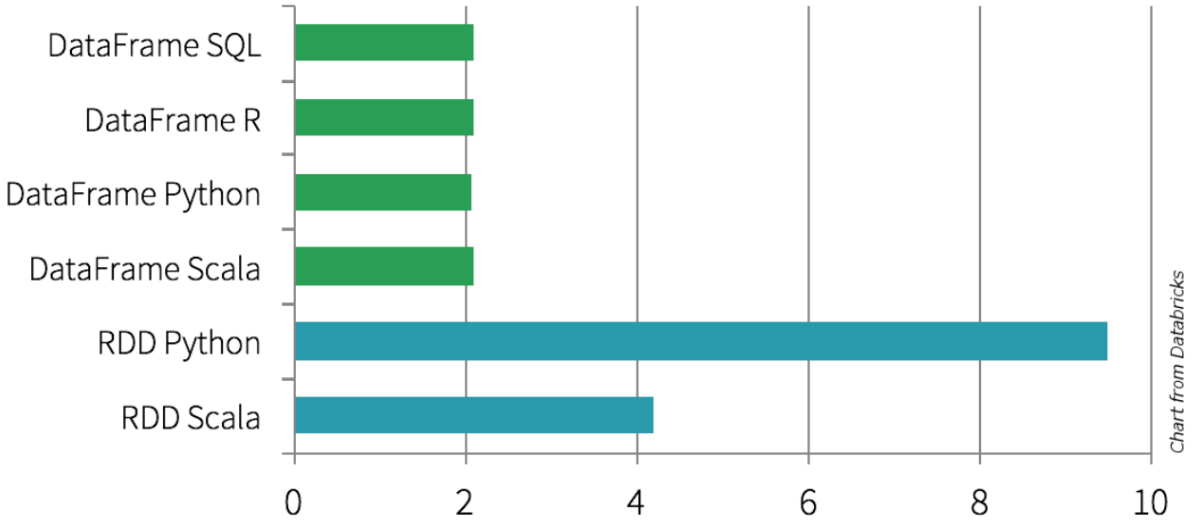
DATAFRAMES/DATASETS (2/2)

- Colección distribuida de datos agrupadas en columnas con nombre
  - RDD con esquema de datos (SchemaRDD)
- Funciones específicas de dominio diseñadas para las tareas comunes
  - Metadata
  - Sampling
  - Filtrado, Agregación, Unión, …
  - UDFs

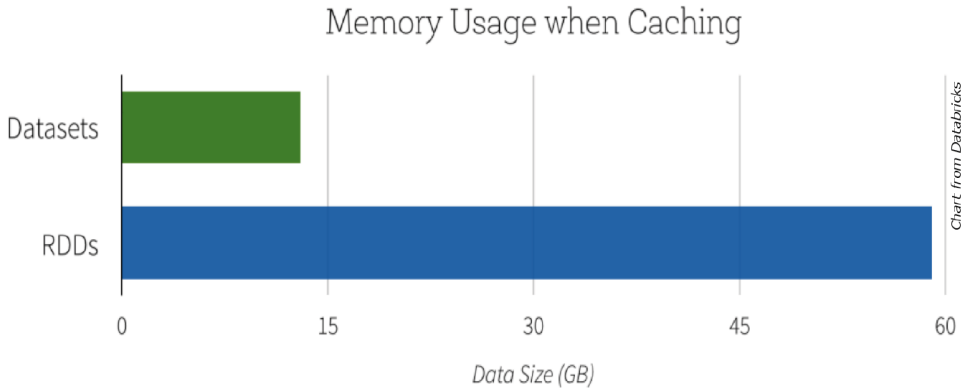


- Disponibles en Python, Scala, Java y R (SparkR)

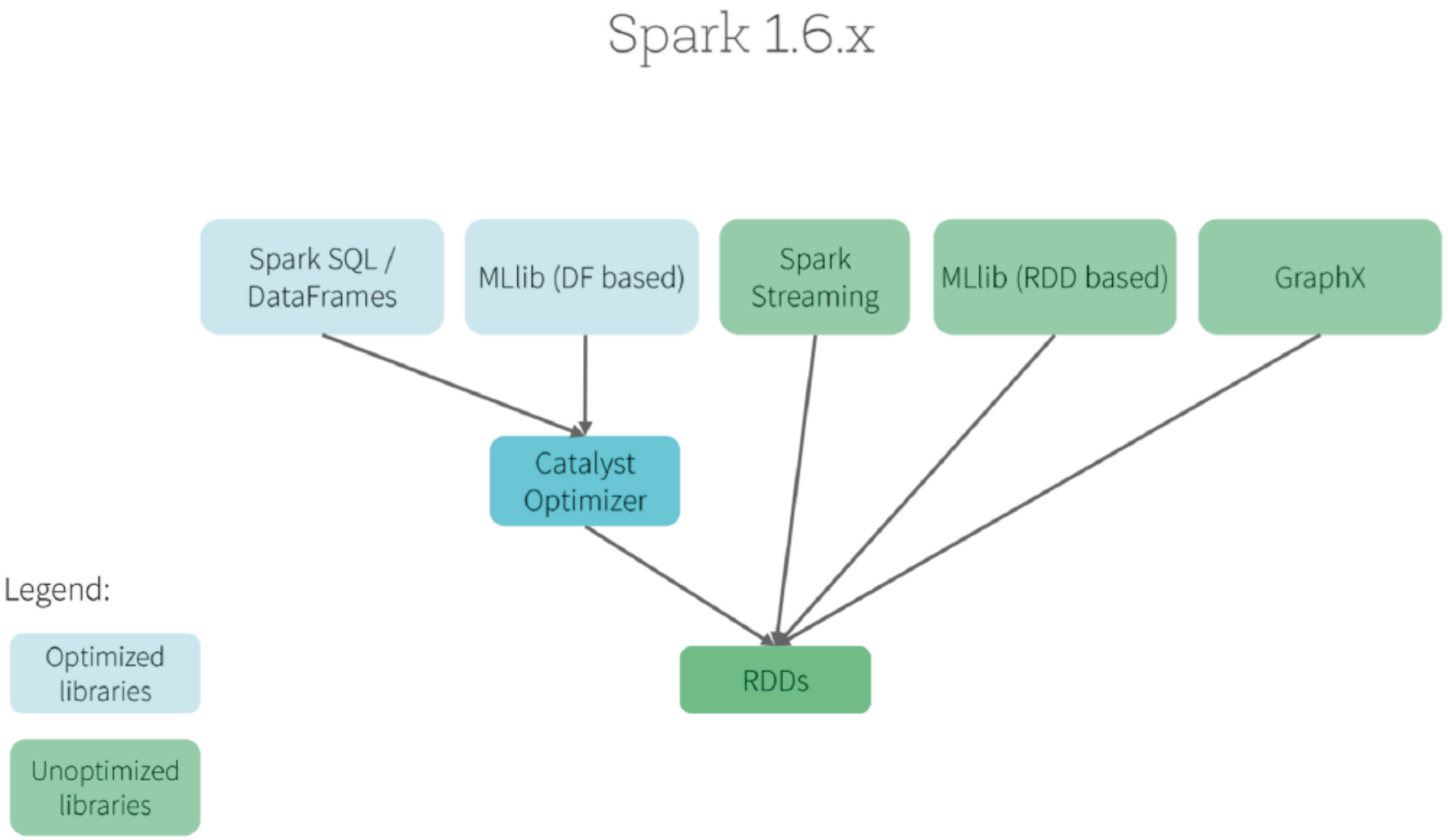
DATAFRAMES/DATASETS: RENDIMIENTO



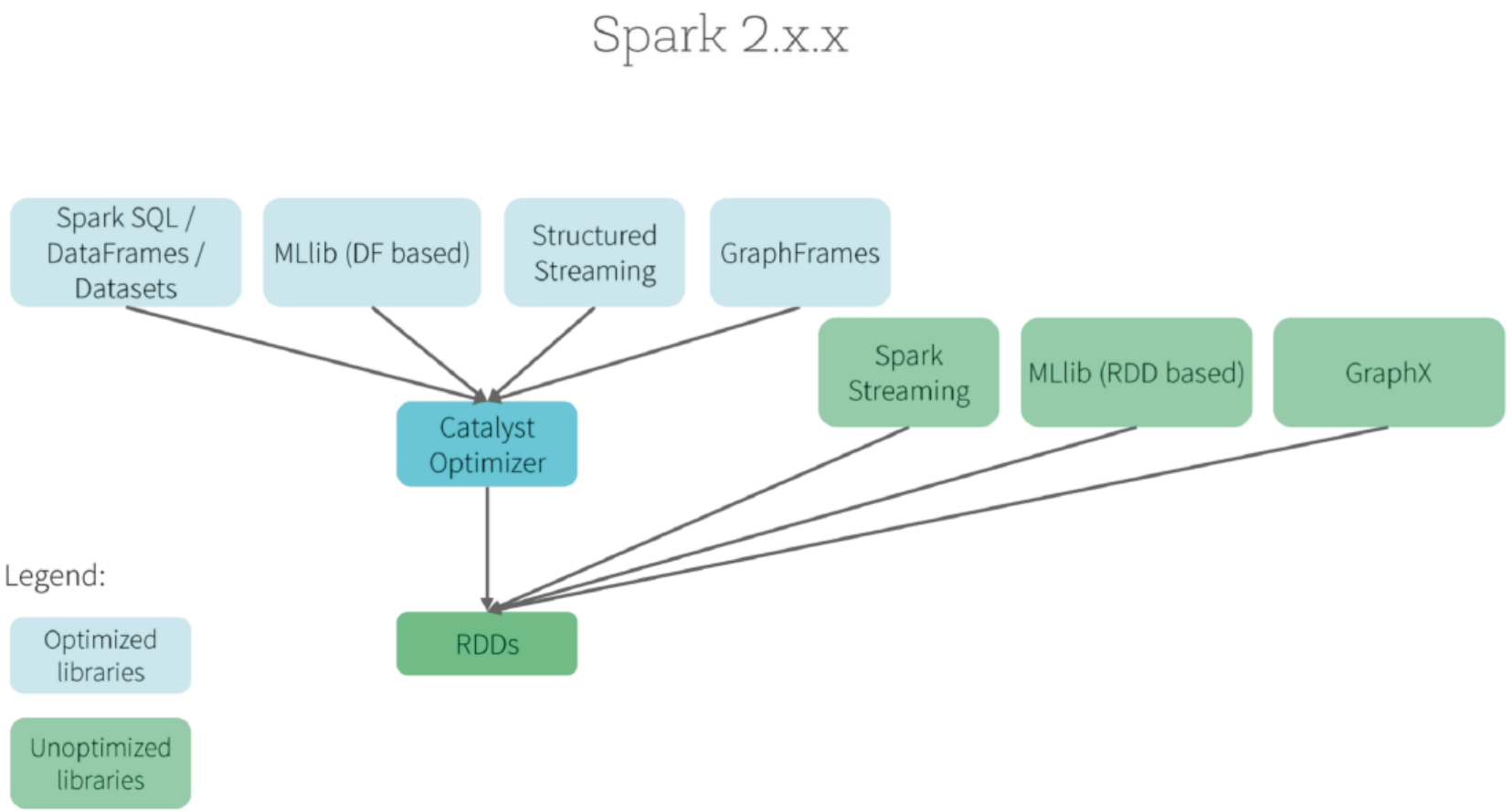
Time to aggregate 10 million integer pairs (in seconds)



DATAFRAMES/DATASETS: OPTIMIZACIÓN Y SOPORTE (1/2)

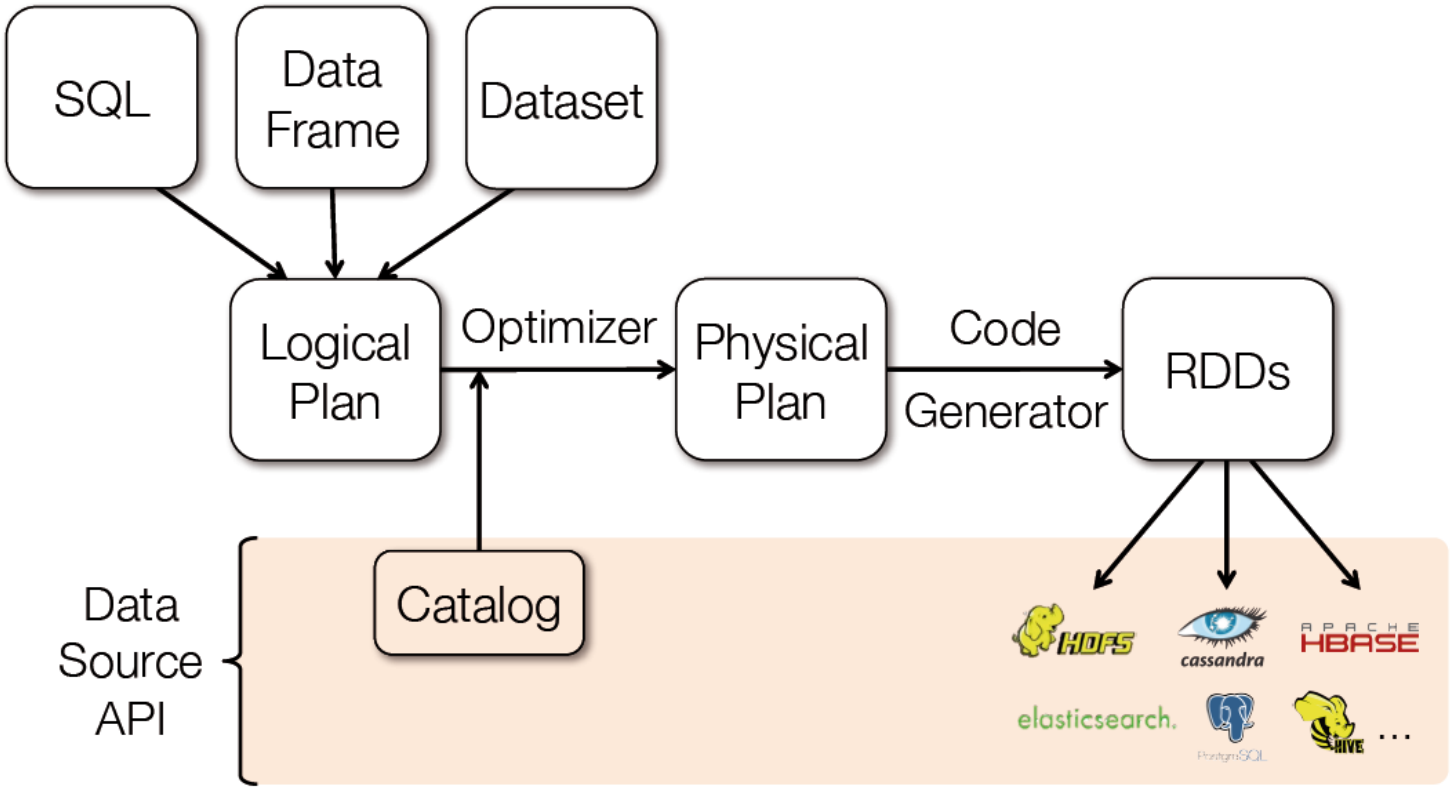


DATAFRAMES/DATASETS: OPTIMIZACIÓN Y SOPORTE (2/2)

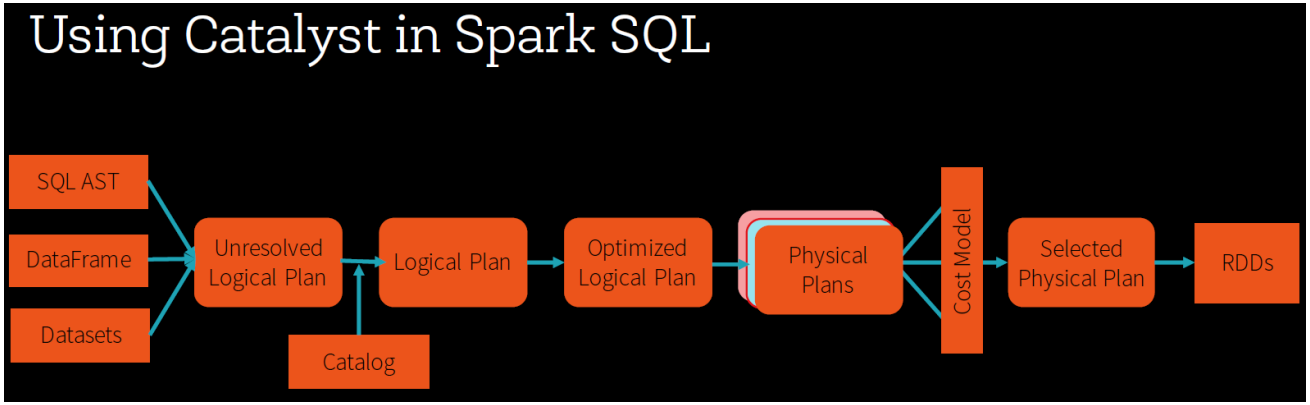


DATAFRAMES/DATASETS: VINCULACIÓN A SPARK SQL

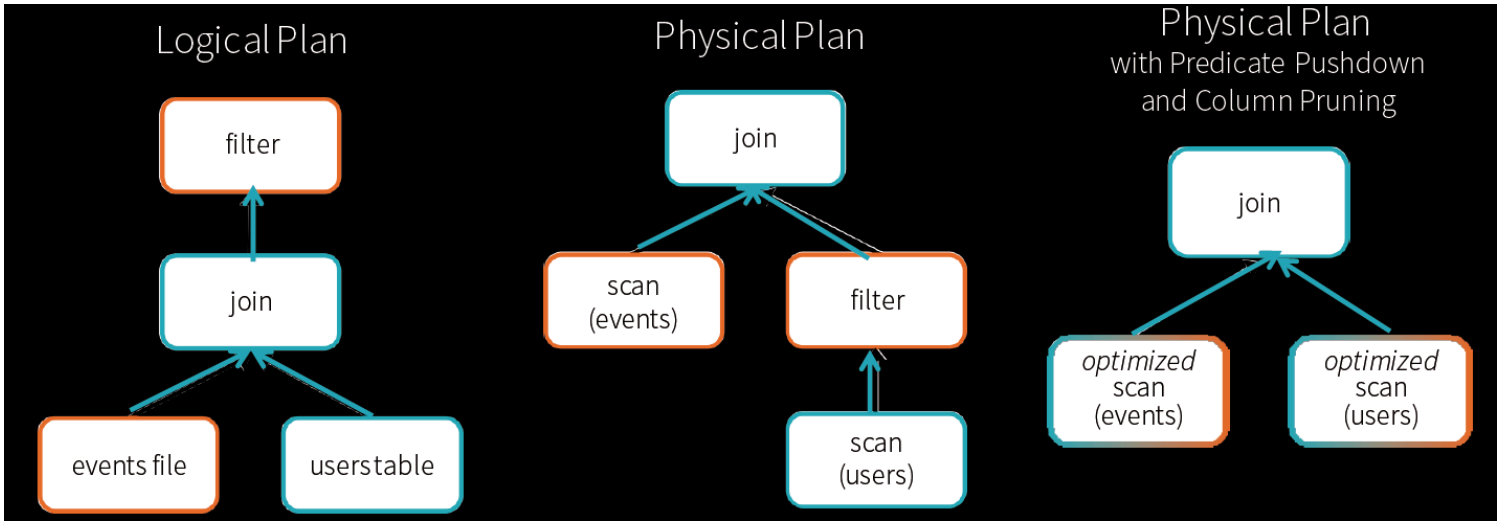
- http://people.csail.mit.edu/matei/papers/2015/sigmod_spark_sql.pdf



EJEMPLO DE OPTIMIZACIÓN CON CATALYST

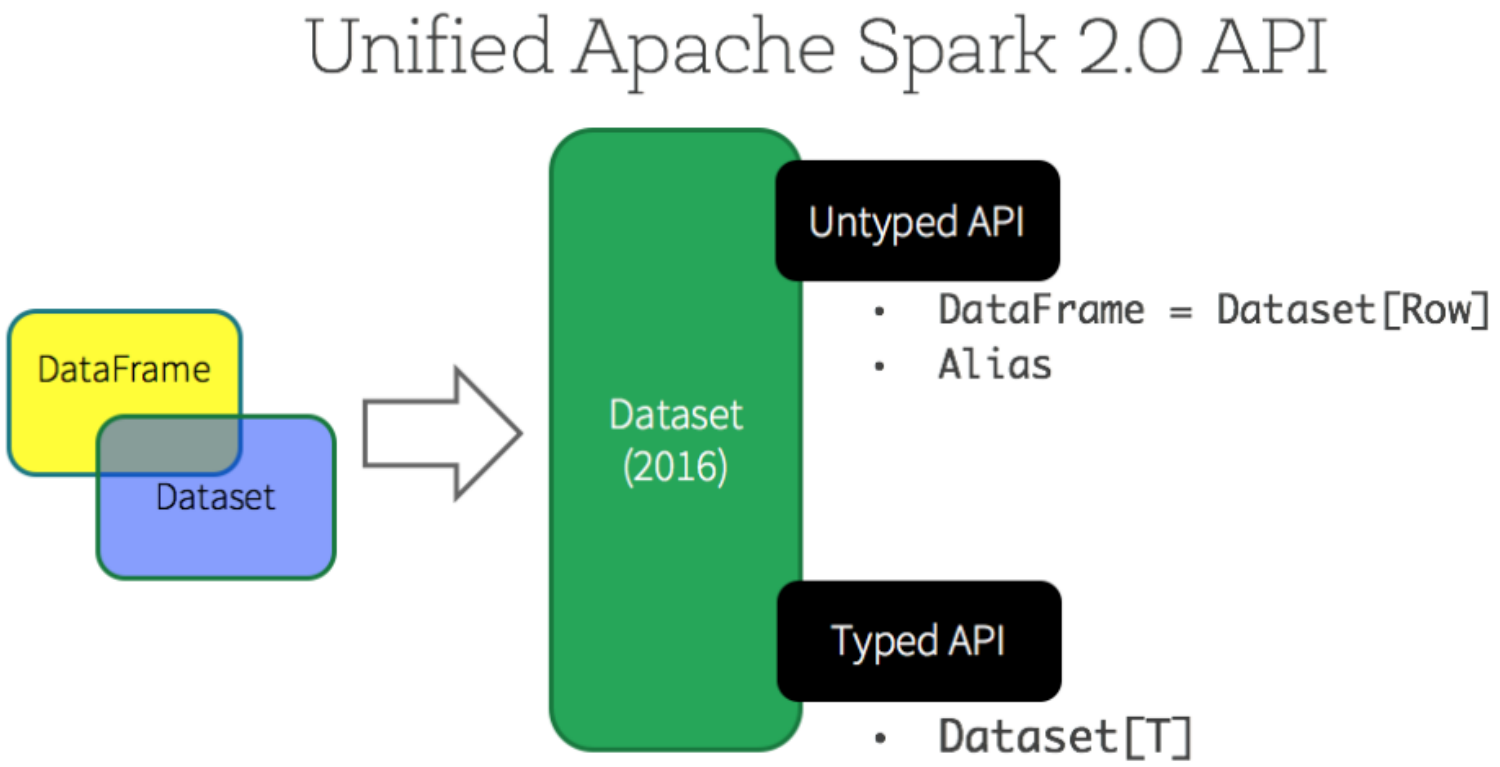


users.join(events, users("id") === events("uid")).filter(events("date") > "2015-01-01")



DATAFRAMES/DATASETS: TIPADO (1/2)

- https://www.kdnuggets.com/2017/08/three-apache-spark-apis-rdds-dataframes-datasets.html



DATAFRAMES/DATASETS: TIPADO (2/2)

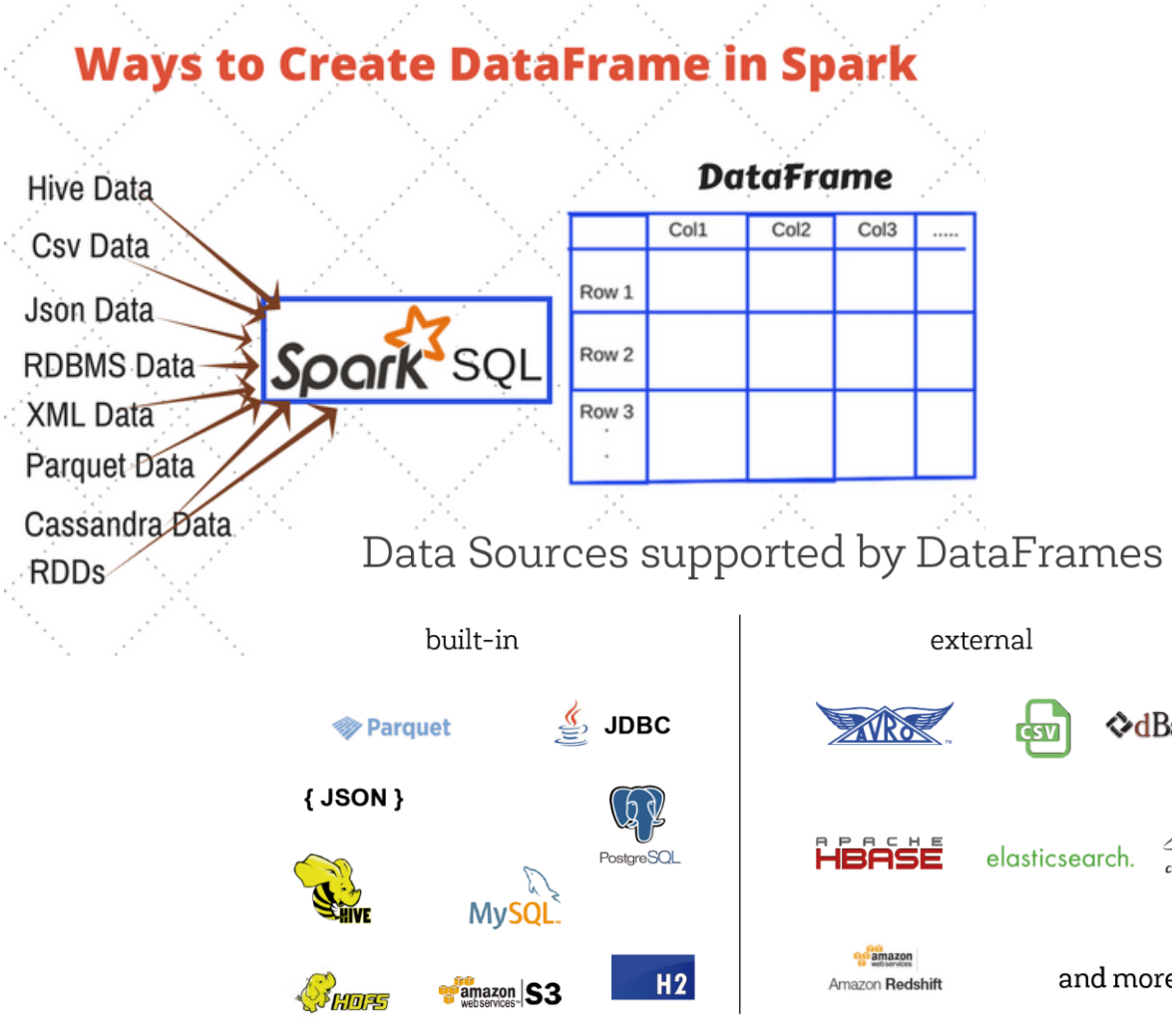
Typed and Un-typed APIs

Language	Main Abstraction
Scala	Dataset[T] & DataFrame (alias for Dataset[Row])
Java	Dataset[T]
Python*	DataFrame
R*	DataFrame

Note: *Since Python and R have no compile-time type-safety, we only have untyped APIs, namely DataFrames.*

- https://spark.apache.org/docs/latest/sql-programming-guide.html

DATAFRAMES/DATASETS: FUENTES DE DATOS



Data Sources

- Connecting to SQL Databases using JDBC
- Amazon Redshift
- Amazon S3
- Amazon S3 Select
- Azure Blob Storage
- Azure Data Lake Storage Gen1
- Azure Data Lake Storage Gen2
- Azure Cosmos DB
- Azure SQL Data Warehouse
- Cassandra
- Couchbase
- ElasticSearch
- Images
- Import Hive Tables
- MongoDB
- Neo4j
- Oracle
- Avro Files
- CSV Files
- JSON Files
- LZO Compressed Files
- Parquet Files
- Redis
- Riak Time Series
- Snowflake
- Zip Files

DATASET DESDE JSON

```
val df = spark.read.json("databricks/people.json")

case class Person(id: BigInt, age: BigInt, name: String , city: String, occupation: String)

import org.apache.spark.sql.Dataset

val ds: Dataset[Person] = df.as[Person]

val filterDS = ds.filter ( p=> p.age > 1000)

filterDS.collect()
```